(which is common to all threads) could check its own thread ID against the stored value:

```
std::thread::id master_thread;
void some_core_part_of_algorithm()
{
    if(std::this_thread::get_id()==master_thread)
    {
        do_master_thread_work();
    }
    do_common_work();
}
```

Alternatively, the `std::thread::id` of the current thread could be stored in a data structure as part of an operation. Later operations on that same data structure could then check the stored ID against the ID of the thread performing the operation to determine what operations are permitted/required.

Similarly, thread IDs could be used as keys into associative containers where specific data needs to be associated with a thread and alternative mechanisms such as thread-local storage aren't appropriate. Such a container could, for example, be used by a controlling thread to store information about each of the threads under its control or for passing information between threads.

The idea is that `std::thread::id` will suffice as a generic identifier for a thread in most circumstances; it's only if the identifier has semantic meaning associated with it (such as being an index into an array) that alternatives should be necessary. You can even write out an instance of `std::thread::id` to an output stream such as `std::cout`:

```
std::cout<<std::this_thread::get_id();
```

The exact output you get is strictly implementation dependent; the only guarantee given by the standard is that thread IDs that compare as equal should produce the same output, and those that are not equal should give different output. This is therefore primarily useful for debugging and